The Lakehouse That Finally Embraces the Database

Graziano Montanaro

PyCon IT 2026 | Bologna | 29 May

Intella

THE PROBLEM

How Did We End Up Here?

Manifests all the way down

Iceberg: metadata JSON → manifest lists → manifests → data files.
Delta: JSON transaction logs.
All of these need compaction eventually.

A catalog server just to read metadata

Hive Metastore, Nessie, or Unity Catalog running 24/7 so your queries know which columns exist.

S3 listing is not a database

File-based catalogs inherit object storage consistency issues: phantom reads, stale manifests, race conditions on concurrent writes.

Spark to VACUUM Spark's mess

You need a JVM cluster to compact the metadata files your JVM cluster created. For tables that fit in a laptop's RAM.

"What if we managed table metadata in a simple database instead of a complex file system?"

INTRODUCTION

What is DuckLake?

SQL Catalog

Metadata in PostgreSQL, SQLite, or DuckDB. No manifest files on S3, no file listing.

Parquet Data Files

Immutable Parquet on any object store or local disk. Same data format everyone already uses.

Real ACID Transactions

Multi-table, multi-statement transactions with snapshot isolation. It's just your catalog DB doing what DBs do.

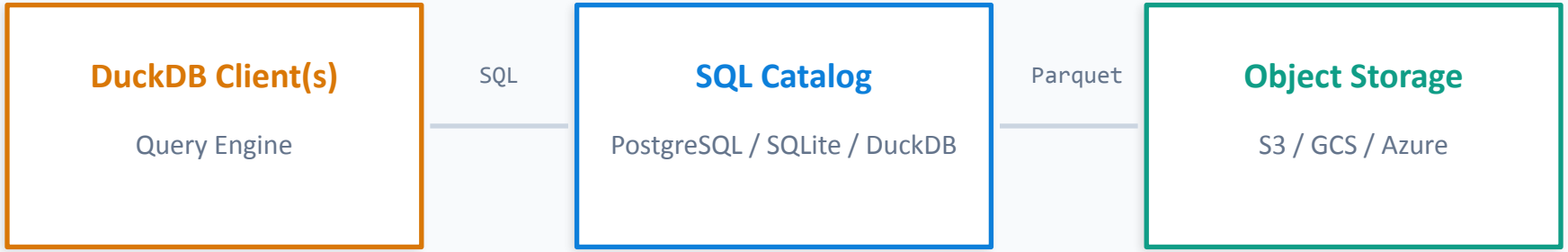
Time Travel & CDC

Query any past snapshot. Built-in change data feed between versions. No Kafka or CDC pipeline.

MIT Licensed | By Raasveldt & Mühleisen (DuckDB Labs) | Announced May 2025 | v1.0 April 2026
Iceberg-compatible data files

ARCHITECTURE

How DuckLake Works



Key Insight

- Metadata lookups are SQL queries against an indexed DB, not file-listing operations on S3
- Data files are immutable Parquet: no lock contention, trivial to cache and replicate
- Multiple DuckDB clients connect to the same catalog concurrently (PostgreSQL or SQLite)

ADOPTION & COMMUNITY

Who's Using It? What's the Reception?

MotherDuck

Managed DuckLake-as-a-service. Full GA support for v1.0 with shared catalogs.

PostHog

Product analytics. Evaluating DuckLake for their analytical data warehouse.

Ascend.io

Data pipeline orchestration. DuckLake integration for simplified lakehouse setup.

Definite

BI platform built entirely on DuckDB + DuckLake. All-in on the "Duck stack."

Community Reception

The fans: *"SQL catalog is an obvious idea in hindsight. Why were we parsing JSON manifests?"*

The skeptics: *"XKCD 927. Iceberg v3 may close the gap before DuckLake gets traction."*

The pragmatists: "Great for terabyte-scale single-team workloads. Not yet replacing Iceberg at hundreds of petabytes."

KEY FEATURE

Data Inlining: The Small Files Killer

How It Works

- Small inserts go straight into the catalog DB instead of creating a Parquet file per batch
- Reads merge inlined data with Parquet transparently: queries see one unified table
- When inlined data grows large enough, it gets flushed to Parquet automatically
- This is what makes streaming ingestion practical without ending up with 10,000 tiny files

Heads up: Inlining is ON by default in v1.0 (threshold: 10 rows). Works on DuckDB, PostgreSQL and SQLite catalogs. Only VARIANT-column inlining is DuckDB-catalog-only.

KEY FEATURE

Data Inlining: The Small Files Killer

How It Works

- Small inserts go straight into the catalog DB instead of creating a Parquet file per batch
- Reads merge inlined data with Parquet transparently: queries see one unified table
- When inlined data grows large enough, it gets flushed to Parquet automatically
- This is what makes streaming ingestion practical without ending up with 10,000 tiny files

Benchmark: 30K Streaming Inserts

926x

faster queries vs. Iceberg

105x

faster ingestion (zero S3 writes)

Iceberg writes ~4 S3 files per batch
(data + manifest + manifest-list + metadata)

Source: ducklake.select/2026/04/02/data-inlining-in-ducklake

Heads up: Inlining is ON by default in v1.0 (threshold: 10 rows). Works on DuckDB, PostgreSQL and SQLite catalogs. Only VARIANT-column inlining is DuckDB-catalog-only.

LIVE DEMO 1

Production Setup: PostgreSQL + S3

SQL

```sql
INSTALL ducklake; INSTALL postgres;
LOAD ducklake;    LOAD postgres;
-- Attach: PostgreSQL catalog + S3 data path
ATTACH ducklake:postgres:host=catalog.prod.internal
  dbname=ducklake_catalog user=lake_admin password=****
AS prod_lake (DATA_PATH 's3://acme-data-lake/warehouse/');
USE prod_lake;
-- Ingest raw Parquet into a managed DuckLake table
CREATE TABLE orders AS
  SELECT * FROM read_parquet('s3://acme-raw/orders/*.parquet');
```

CATALOG BACKENDS

Choosing Your Catalog Backend

SQLite

Multi-process local, edge deployments

Strengths

+ Data inlining works
+ Multi-process via attach/detach pattern
+ Lightweight, no server needed

Weaknesses

- Write lock contention under load
- No remote client access
- Retry-based concurrency can stall

DuckDB

Local dev, single user, prototyping

Strengths

+ Fastest catalog queries (in-process)
+ Data inlining works
+ Zero setup: one .ducklake file

Weaknesses

- Single writer only
- No remote access
- Not ideal for production multi-user

PostgreSQL

Production, multi-user, remote clients

Strengths

+ Concurrent writers via MVCC
+ HA, backups, monitoring: well-understood
+ Remote clients over the network

Weaknesses

- **VARIANT-column inlining unsupported**
- Requires managing a PG instance
- Explicit DATA_PATH required

⚠ MySQL is technically supported but has known connector issues. The DuckLake docs explicitly recommend against it for now.

COMPARISON

DuckLake vs Iceberg vs Delta Lake

	DuckLake	Apache Iceberg	Delta Lake
Metadata Store	SQL Database	Manifest files (Avro)	JSON log files
Catalog Server	Not needed (it IS the DB)	Required (Hive/Nessie)	Optional (Unity)
Data Inlining	Yes (small file killer)	No	No
Multi-Table ACID	Yes (inherited from DB)	Limited (REST catalogs)	Limited (Unity)
Multi-Engine	DuckDB, experimental support for Spark, Trino, DataFusion, Pandas	Spark, Flink, Trino...	Spark, Flink, Trino...
Concurrency	Optimistic (DB-level)	Mature OCC	OCC + log replay
Governance / ACLs	DB permissions only	Rich (REST catalog)	Rich (Unity Catalog)
Scale Tested	TB-scale; millions of snapshots (FAQ)	Petabyte-proven	Petabyte-proven
Setup Complexity	Minimal (2 SQL cmds)	High	Medium

The real differentiator: multi-table ACID and zero-infrastructure metadata.

LIVE DEMO 2

ACID Transactions

SQL

```sql
-- Atomic multi-statement transactions
BEGIN TRANSACTION;

INSERT INTO orders VALUES
  (9001, 'CUST-42', 'laptop',  1499.99, '2026-04-10'),
  (9002, 'CUST-42', 'charger',   49.99, '2026-04-10');

UPDATE orders SET amount = amount * 0.9   -- 10% discount
  WHERE order_id IN (9001, 9002);

COMMIT;  -- Both INSERT and UPDATE apply atomically

-- If anything fails, nothing is written
-- Every committed transaction = a new snapshot in the catalog
```

LIVE DEMO 3

Time Travel & Change Data Feed

SQL

```sql
-- List all snapshots (each write = one snapshot)
SELECT * FROM prod_lake.snapshots();
-- snapshot_id | snapshot_time       | changes
-- 1           | 2026-04-10 09:00    | CREATE TABLE orders
-- 2           | 2026-04-10 09:01    | INSERT 1.2M rows
-- 3           | 2026-04-10 10:00    | INSERT + UPDATE
-- Query data as it was at snapshot 2 (before discount)
SELECT order_id, amount FROM orders AT (VERSION => 2)
WHERE order_id = 9001;   -- 1499.99 (original price)
-- Built-in CDC: diff two snapshots
SELECT * FROM prod_lake.table_changes('orders', 2, 3);
-- change_type | order_id | amount
-- INSERT      | 9001     | 1349.99
```

ROADMAP

v1.0 Shipped. What's Next?

SHIPPED IN v1.0

Data Inlining (Insert / Delete / Update)

ON by default, threshold 10 rows. Works on DuckDB, PostgreSQL and SQLite catalogs.

Multi-Engine Clients & New Types

Spark, DataFusion, Trino, Pandas connectors. GEOMETRY, VARIANT types. Bucket partitioning (Iceberg-compatible).

Sorted Tables + Deletion Vectors

SET SORTED BY for scan-optimised tables. Experimental Puffin-file deletion vectors.

STILL ON THE ROADMAP

v1.1 (Sep 2026): VARIANT Inlining

VARIANT inlining on PG/SQLite catalogs.
Multi-deletion-vector Puffin files (preserve time travel while minimising small files).

v2.0: Git-like Branching

Branch your DuckLake, merge changes. Quack-served DuckDB-as-catalog is already experimental today (PR #1151).

v2.0: Roles, ACLs, Incremental Mat. Views

Permission-based roles, fine-grained access control, and materialised views refreshed incrementally via the change feed.

v1.0 is production-ready with backward-compatibility guarantees. Roadmap: ducklake.select/roadmap

COMING SOON

Quack: DuckDB Goes Client-Server

What Is Quack?

- HTTP-based protocol: DuckDB talks to DuckDB over the network
- Multiple concurrent writers to the same database
- Single round-trip queries, faster than Arrow Flight SQL
- Bulk transfer: 60M rows in under 5 seconds
- Auth via pluggable callbacks (LDAP, tokens, custom)

What It Unlocks for DuckLake

DuckDB as Catalog Server

A remote DuckDB instance serves as the DuckLake catalog. No PostgreSQL needed for multi-user.

Inlining + Concurrency

Inlining now works on PG and SQLite too. Quack additionally enables DuckDB-as-remote-catalog (experimental in DuckLake PR #1151).

Wasm Support

DuckDB in a browser connects directly to a Quack server. Interactive lakehouse dashboards without a backend.

Timeline

Released May 12, 2026
Quack beta ships in DuckDB v1.5.2 via core_nightly

September 2026
Quack GA together with DuckDB v2.0

Already experimental
DuckLake-as-Quack-catalog: PR #1151 in duckdb/ducklake

HONEST ASSESSMENT

The Fine Print

"Isn't this just Hive Metastore?"

Both put metadata in a DB, yes. But DuckLake is a full table format spec with ACID, time travel, schema evolution. HMS is a metadata service. Still, expect to hear this comparison a lot.

VARIANT inlining: DuckDB-catalog only

Inlining of VARIANT semi-structured columns requires DuckDB as catalog. PostgreSQL and SQLite cannot round-trip VARIANT through string. All other inlining (inserts, deletes, updates) works on PG and SQLite in v1.0.

Catalog DB is now critical infrastructure

Optimistic concurrency handles moderate writes; many parallel writers can exhaust retries. Your catalog DB needs HA, backups, monitoring: the usual ops burden.

Governance and tooling are not there yet

No fine-grained ACLs, no data lineage, no dbt/Airflow/Fivetran plugins. Access control means DB-level GRANTs. Enterprise-grade governance is still on the roadmap.

Sources: ducklake.select/docs/stable | ducklake.select/manifesto | ducklake.select/2026/04/13/ducklake-10

Three Stacks, Three Trade-offs

PySpark + Iceberg

Strengths

+ Petabyte-scale proven
+ Multi-engine (Spark, Flink, Trino)
+ Rich governance (REST catalog)
+ Massive community & tooling

Weaknesses

- Infra-heavy: JVM, YARN/K8s, Metastore
- Compaction jobs required
- Slower for small-medium data

Polars + Delta Lake

Strengths

+ Fast single-node performance
+ Python-native, no JVM
+ Delta is battle-tested
+ Growing Rust ecosystem

Weaknesses

- No native Delta write (need delta-rs)
- **delta-rs deletion vectors: read-only (writes/vacuum open)**
- Limited to single-node scale

DuckDB + DuckLake

Strengths

+ Simplest setup (2 SQL commands)
+ Data inlining solves small files
+ Built-in time travel + CDC
+ Multi-table ACID transactions

Weaknesses

- Multi-engine connectors still new
- v1.0 just released, less tested
- No fine-grained governance
- **VARIANT inlining needs DuckDB catalog**

No single winner. Pick the stack that fits your team and your data volume.

DECISION FRAMEWORK

Should You Actually Use This?

Use DuckLake When...	Don't Use DuckLake When...
✓ Your data is in the terabyte range (FAQ: no practical limit; catalog DB-bound)	✗ You need Flink, Polars, or Ray reading the same tables
✓ Small team, no dedicated infra engineers	✗ Petabyte-scale with hundreds of concurrent writers
✓ DuckDB is already your go-to analytics engine	✗ Fine-grained governance and data lineage are non-negotiable
✓ You want time travel + CDC without spinning up Kafka	✗ PySpark + Iceberg is already working for you. Don't fix it.
✓ Local dev first, promote to cloud when ready	✗ You need fine-grained row/column security at the lakehouse layer (v2.0)
✓ Prototyping a lakehouse before committing to Iceberg	✗ Compliance requires a battle-tested production track record

DuckLake is not trying to replace Iceberg. It is a simpler tool for a different (smaller) problem.

SUMMARY

Key Takeaways

1

A SQL database beats file-listing for metadata

For small-to-medium workloads, querying an indexed catalog is 10-100x faster than parsing JSON manifests on S3. And it's a lot easier to debug when something goes wrong.

2

One process. One pip install. The whole lakehouse.

pip install duckdb, attach Postgres for the catalog, point at S3. ACID, time travel, CDC, schema evolution. No Spark, no JVM, no YARN.

3

Know the boundaries

Per the docs: terabytes of data and millions of snapshots, catalog-DB bound. At hundreds of petabytes with many concurrent engines, Iceberg's ecosystem still wins. Pick the tool that matches your actual workload.

Code demo

DuckLake

The Lakehouse That Finally Embraces the Database

One database for metadata. One bucket for data. That's the whole lakehouse.

GitHub
`github.com/montanarograziano`

Website
`ducklake.select`

Docs
`ducklake.select/docs/stable`

LinkedIn

Questions?

Graziano Montanaro | graziano.montanaro@intella.tech

Intella